

Relatório de Critérios de Aceite — Criação do MCP do Projeto

Ticket GLPI #10350 · Evidências ad-hoc (sem UI do SigSUAS)

Gerado em: 07/08/2026

Repositório: pwdev-solucoes/hu-generator-suas-mcp (privado), submódulo em .claude/mcp/hu-generator-suas

Ambiente de conexão do Agente: Claude Desktop

Ticket GLPI: #10350 — HU - #10350 Criação do MCP do Projeto

Commit avaliado: a5b0e3e

Ferramenta: node scripts/test-client.mjs (cliente MCP real via stdio) + npm/git em terminal — sem playwright/navegador, este ticket não envolve tela do SigSUAS

Resumo

6 critérios de aceite, todos verificados por execução real de comando (instalação limpa, script de atualização, cliente MCP de teste ponta-a-ponta) — nenhum via navegador, já que o escopo do ticket é o servidor MCP e sua integração com o Claude Desktop, não uma tela do SigSUAS. 6/6 atendidos.

CA	CRITÉRIO	STATUS
CA01	Conectar o MCP ao Claude Desktop	ATENDIDO · automatizado
CA02	Consultar a base de conhecimento via tools	ATENDIDO · automatizado
CA03	Repositório autocontido, pronto para instalar	ATENDIDO · automatizado
CA04	Atualizar com um comando	ATENDIDO · automatizado
CA05	Pacote pronto para uso como Claude Project	ATENDIDO · automatizado
CA06	Acesso restrito só ao conteúdo autorizado	ATENDIDO · automatizado

CA01 — Conectar o MCP ao Claude Desktop ATENDIDO · automatizado

Critério (Gherkin): Dado o repositório hu-generator-suas-mcp instalado localmente, quando o usuário configura o servidor no `claude_desktop_config.json` (macOS, Linux ou Windows) e reinicia o Claude Desktop, então o servidor aparece conectado e suas tools/resources ficam disponíveis em qualquer conversa.

Como foi testado: O README documenta o caminho exato de `claude_desktop_config.json` para macOS/Linux/Windows e o JSON de configuração (com barras invertidas escapadas na variante Windows). O mesmo mecanismo de conexão (`stdio`, `StdioClientTransport`) que o Claude Desktop usa foi exercitado por `scripts/test-client.mjs`, que abre o processo `node dist/index.js` via `stdio` e conecta como um cliente MCP real.

Resultado observado:

- Saída real do teste: `"=== connected ==="` seguido de `"=== resources/list: 88 recursos ==="` e `"=== tools/list ==="` listando as 5 tools — confirma que a conexão `stdio` (o mesmo transporte usado pelo Claude Desktop) expõe corretamente tools e resources assim que o servidor sobe.

Evidência por cobertura automatizada — ver Anexo.

CA02 — Consultar a base de conhecimento via tools ATENDIDO · automatizado

Critério (Gherkin): Dado o MCP conectado, quando o Agente usa as tools `list_modules`, `search_modules`, `get_module`, `get_business_rule` ou `lookup_glossary`, então recebe o conteúdo correspondente (módulos, regras de negócio, glossário) sem precisar copiar/colar arquivo.

Como foi testado: Execução de `node scripts/test-client.mjs` chamando as 5 tools em sequência contra o conteúdo empacotado real.

Resultado observado:

- `tools/list` retornou exatamente: `list_modules`, `search_modules`, `get_module`, `get_business_rule`, `lookup_glossary`.
- `list_modules(frontend='client', status='entregue')` retornou linhas reais do inventário (ex.: `INV-003 · social-units`, `INV-004 · families`, `INV-005 · address-inconsistencies`).
- `search_modules(query='LGPD')` retornou trecho da tabela condensada + trechos de fichas completas (`address-inconsistencies`, `geo-panel`).
- `get_module(slug='families')` retornou a ficha completa com citação de fonte real (Model + migrations).
- `get_business_rule(id='RN-domain-01')` retornou o texto da regra de negócio real.
- `lookup_glossary(term='CRAS')` retornou as entradas de glossário CRAS e PAIF.

Evidência por cobertura automatizada — ver Anexo.

CA03 — Repositório autocontido, pronto para instalar ATENDIDO · automatizado

Critério (Gherkin): Dado um clone limpo do repositório do MCP, quando o usuário instala as dependências, então o servidor já está pronto para rodar (build já versionado, conteúdo já empacotado), sem precisar de outro checkout do projeto nem de build manual.

Como foi testado: Simulação de clone limpo no checkout real: `rm -rf node_modules dist` (remove tudo que `.gitignore` ignora) → `npm install` → `git checkout -- dist` (restaura só o que está versionado, sem rodar build) → roda o cliente de teste direto contra o `dist/` restaurado do git.

Resultado observado:

- Após remover `dist/`, ele não existia mais no disco ("ls: dist: No such file or directory").
- `git checkout -- dist` restaurou `dist/index.js` a partir do git (arquivo rastreado no repositório, confirmado por `git ls-files dist/` → `dist/index.js`).
- `node scripts/test-client.mjs`, usando esse `dist/` restaurado do git (sem rodar `npm run build`), conectou e listou os 88 resources normalmente — confirma que o build versionado funciona sem rebuild.

Evidência por cobertura automatizada — ver Anexo.

CA04 — Atualizar com um comando ATENDIDO · automatizado

Critério (Gherkin): Dado uma nova versão de conteúdo/código publicada no repositório do MCP, quando o usuário roda o comando de atualização, então sua cópia local é atualizada (código + dependências + build), sem passos manuais adicionais.

Como foi testado: Execução de `bash scripts/update.sh` (equivalente a `npm run update`) no checkout real.

Resultado observado:

- Saída real: `"== git pull =="` → `"Already up to date."` → `"== npm install =="` → `"up to date, audited 101 packages"` → `"== npm run build =="` → `"tsc -p ."` → `"Atualizado. dist/index.js reconstruído."` — as 3 etapas (pull, install, build) rodaram em sequência com um único comando.

Evidência por cobertura automatizada — ver Anexo.

CA05 — Pacote pronto para uso como Claude Project ATENDIDO · automatizado

Critério (Gherkin): Dado um usuário que quer criar um Project no Claude.ai com o mesmo conhecimento, quando ele usa os arquivos da pasta `claude-project/` do MCP, então consegue configurar o Project tanto com o MCP conectado (fonte viva) quanto sem MCP (arquivo de conhecimento estático anexado).

Como foi testado: Inspeção do diretório `claude-project/` no repositório real.

Resultado observado:

- Pasta contém os 3 arquivos esperados: `README.md` (explica os dois modos de uso + instalação), `claude-project-instructions.md` (v2, com seção 'FONTE DE CONHECIMENTO' que detecta MCP conectado ou não) e `claude-project-knowledge.md` (v2, fallback estático).
- `README.md` da raiz do repositório referencia `claude-project/` na seção 'Usar como Claude Project'.
- Nenhuma referência a 'esuas-all' ou 'eSUAS/SigSUAS' nos 3 arquivos (grep -c retornou 0 em todos), consistente com o repositório ser distribuível de forma independente.

Evidência por cobertura automatizada — ver Anexo.

CA06 — Acesso restrito só ao conteúdo autorizado ATENDIDO · automatizado

Critério (Gherkin): Dado uma tentativa de leitura de arquivo fora dos diretórios de conhecimento expostos pelo MCP, quando essa tentativa acontece através de uma `tool/resource`, então o servidor recusa o acesso.

Como foi testado: `get_module(slug='../././SKILL')` via `scripts/test-client.mjs` — tentativa de path traversal usando `../` para escapar do diretório de conhecimento exposto.

Resultado observado:

- Resultado real: "Acesso negado: \"../././SKILL\" resolve para um caminho fora do que este servidor expõe." — a tentativa foi rejeitada com mensagem de acesso negado, sem expor nenhum arquivo fora do escopo autorizado (`knowledge/catalog/templates/checklists`).

Evidência por cobertura automatizada — ver Anexo.